Lista de Exercícios – JavaScript e jQuery: Projeto Comentários

Instituto Federal do Piauí – Campus Paulistana ·

Professor: Prof. Me. Julian Rodrigues Valério · Disciplina: _____ · Turma: _____

Aluno(a): _____ · Data: ____ / ____ / _____

Ao concluir esta lista, você deverá ser capaz de explicar cada linha do arquivo `comentarios.js`: o que a sintaxe significa, o que o jQuery faz e por que a lógica funciona.

Objetivos de aprendizagem

- Explicar como o HTML carrega o jQuery e o script, e quando o código começa a executar.
- Reconhecer a sintaxe usada: `var`, expressões de função, `"use strict"`, comentários, operadores e ponto e vírgula.
- Distinguir selecionar elementos de criar elementos com jQuery e usar encadeamento de métodos.
- Explicar a lógica de validação, criação e inserção de um comentário.
- Explicar eventos, callbacks, `this` e delegação de eventos.
- Rastrear a execução do código, encontrar erros e propor modificações.

Como usar esta lista

1. Mantenha `index.html`, `style.css` e `comentarios.js` abertos e a página aberta no navegador.
2. Use o console do navegador (tecla F12, aba Console) para testar e ler mensagens de erro.
3. Quando um exercício citar uma linha, use a numeração do código de referência abaixo.

Código de referência – `comentarios.js` numerado

```
01  var main = function(){
02      //window.alert("Hello World");
03      "use strict";
04      var $input = $(".comment-input input")
05      var $comments = $(".comments");
06
07      var criarComentario = function(texto){
08          var $comentario = $("<p>").addClass("comment");
09          var $texto = $("<span>").text(texto);
10          var $botaoExcluir = $("<button>")
11              .addClass("delete-comment")
12              .attr("type", "button")
13              .text("Excluir");
14          $comentario.append($texto, " ", $botaoExcluir);
15          return $comentario;
16      }
17
18      var addCommentFromInputBox = function(){
19          var texto = $input.val().trim(); //bloqueia espaços vazios
20          if(texto === ""){
21              $input.val("").focus();
```

```
22              return;
23          }
24          var $novoComentario = criarComentario(texto);
25          $novoComentario.hide();
26          $comments.append($novoComentario);
27          $novoComentario.fadeIn();
28          $input.val("").focus();
29      };
30      $comments.on("click", ".delete-comment", function(){
31          $(this).closest(".comment").fadeOut( function(){
32              $(this).remove();
33          });
34      });
35
36      $(".comment-input button").on("click", function (event){
37          addCommentFromInputBox();
38      });
39      $(".comment-input input").on("keypress", function(event){
40          if(event.keyCode === 13){
41              addCommentFromInputBox();
42          }
43      });
44      /*$(".comment-input input").on("keypress", function(event){
45          console.log("Código da tecla "+ event.keyCode);
46      });*/
47  };
48  $(document).ready(main);
```

Parte 1 – Estrutura geral e carregamento da página

Antes de entender o JavaScript, é preciso saber como e quando ele começa a executar.

1. [Análise] No `index.html`, quais são as duas tags `<script>` e em que ordem aparecem? Explique o que aconteceria se `comentarios.js` fosse carregado antes do jQuery. Qual mensagem de erro apareceria no console?

2. [Conceitual] Por que as tags `<script>` estão no final do `<body>`, e não no `<head>`? Relacione sua resposta com o momento em que os elementos HTML passam a existir para o navegador.

3. [Análise] Explique a linha 48, `$(document).ready(main);`. Qual é a diferença entre `$(document).ready(main)` e `$(document).ready(main())`? Qual das duas está correta e por quê?

4. [Conceitual] A URL do jQuery usa `http://`. O que pode acontecer se a página for publicada em um endereço `https://`, como o GitHub Pages? Proponha a correção.

5. [Análise] Complete a tabela com o elemento HTML encontrado por cada seletor usado em `comentarios.js` e indique se ele já existe no `index.html` ou se é criado pelo JavaScript.

Seletor	Linha(s)	Elemento HTML encontrado	Existe no HTML ou é criado pelo JS?
`.comment-input input`	4, 39		

Seletor	Linha(s)	Elemento HTML encontrado	Existe no HTML ou é criado pelo JS?
`.comments`	5		
`.comment-input button`	36		
`.delete-comment`	11, 30		
`.comment`	8, 31		

Parte 2 – Sintaxe do JavaScript

Esta parte trata da linguagem em si, independentemente do jQuery.

7. [Análise] Na linha 1, `var main = function(){ ... };`, identifique: a palavra-chave, o identificador, o operador e o tipo de valor atribuído. Qual é o nome técnico dessa forma de criar uma função? Reescreva-a como declaração de função, começando com `function main()`.

8. [Conceitual] Qual é a diferença entre os comentários `//` (linhas 2 e 19) e `/* ... */` (linhas 44 a 46)? O que as linhas 44 a 46 fariam se fossem descomentadas, e por que provavelmente foram mantidas no arquivo?

9. [Teste] A linha 3 contém `"use strict";`.

- a) O que essa instrução faz e qual é o seu alcance (o arquivo inteiro ou só a função `main`)?
- b) Por que ela é escrita entre aspas, como se fosse um texto?
- c) Ela teria efeito se fosse colocada depois da linha 4? Justifique.
- d) Dentro de `main`, escreva `x = 10;` sem declarar `x`. Recarregue a página com e sem `"use strict"` e registre o que acontece em cada caso.

10. [Análise] A linha 4 não termina com ponto e vírgula, mas a linha 5 termina. O código funciona mesmo assim? Explique o mecanismo do JavaScript que permite isso e por que é recomendável padronizar o uso do `;`.

11. [Análise] Compare o final da linha 16 (`}`) com o final da linha 29 (`};`). As duas linhas encerram o mesmo tipo de instrução. Qual delas está mais consistente com a linha 1 e por quê?

12. [Conceitual] As variáveis `$input` e `$comments` são declaradas dentro de `main`, fora das funções `criarComentario` e `addCommentFromInputBox`. Explique como essas funções internas conseguem usar essas variáveis. Em seguida, digite `$input` no console do navegador e explique o resultado.

13. [Teste] Copie a chamada `addCommentFromInputBox();` para logo depois da linha 5 e recarregue a página. Qual erro aparece? Explique por que ele acontece, usando o conceito de *hoisting* (içamento). O erro aconteceria se a função fosse criada com `function addCommentFromInputBox(){ ... }`?

14. [Teste] Avalie no console as expressões abaixo e registre os resultados. Depois, explique por que o código usa `===` nas linhas 20 e 40.

Expressão	Resultado
`"" == 0`	
`"" === 0`	
`"13" == 13`	
`"13" === 13`	

15. [Análise] Na linha 19, qual é o tipo do valor devolvido por `$input.val()`? E por `.trim()`? Complete a tabela.

Valor digitado no campo	Resultado de `.trim()`
`"  Olá  "`	
`"     "`	
`""`	
`"a   b"`	

Parte 3 – Seletores e objetos jQuery

O jQuery é uma biblioteca: tudo o que ele oferece passa pela função `$`.

16. [Conceitual] O que é `$` no código? É um símbolo especial da linguagem JavaScript ou um nome comum? Quem o define e o que ele devolve quando recebe um seletor CSS?

17. [Conceitual] Por que as variáveis `$input`, `$comments`, `$comentario` e `$botaoExcluir` começam com `$`, mas a variável `texto` (linha 19) não? Essa regra é obrigatória?

18. [Teste] Execute no console `$("p")` e depois `$("<p>")`. Descreva a diferença entre os dois resultados. Qual dos dois é usado nas linhas 8, 9 e 10, e com qual objetivo?

19. [Análise] No seletor `.comment-input input` (linha 4), qual é o significado do espaço entre `.comment-input` e `input`? O `<input>` do HTML possui um `id`. Reescreva a linha 4 usando esse `id` e explique qual das duas formas é mais direta.

20. [Código] A linha 4 guarda o campo de texto na variável `$input`, mas a linha 39 seleciona o mesmo elemento de novo. Que desvantagens tem repetir a seleção? Reescreva as linhas 36 e 39 reaproveitando variáveis (crie uma variável `$botao` para o botão).

21. [Teste] A linha 21 usa `$input.val("").focus();`. Execute no console `$("#commentInput").val().focus()` (sem o argumento `""`). O que acontece? Explique por que `val("")` permite encadear `.focus()` e `val()` não permite.

Parte 4 – A função criarComentario (linhas 7 a 16)

Esta função monta um comentário completo, mas não o coloca na página.

22. [Análise] Qual é o parâmetro de `criarComentario` e o que ela devolve com `return`? Escreva o HTML exato que a função monta quando é chamada como `criarComentario("Muito bom!")`.

23. [Análise] As linhas 10 a 13 formam uma única instrução. Explique como isso é possível. Para cada método (`addClass`, `attr`, `text`), diga o que ele faz e o que ele devolve, permitindo que o próximo método seja chamado.

24. [Teste] Na linha 9, troque temporariamente `.text(texto)` por `.html(texto)`. Adicione os comentários `<b>negrito</b>` e `<img src=x onerror=alert('invadido')>`. Descreva o que acontece em cada caso e repita o teste com `.text()`. Explique por que `.text()` é a escolha segura para textos digitados pelo usuário.

25. [Teste] A linha 14 chama `append` com três argumentos. Quais são eles e qual é a função da string `" "`? Remova a string, adicione um comentário e observe. Explique o resultado considerando a regra `.comment` do `style.css`.

26. [Conceitual] Por que a linha 12 define `type="button"` no botão? O que poderia acontecer se o comentário estivesse dentro de um `<form>` e essa linha fosse removida?

27. [Conceitual] `criarComentario` cria o elemento, mas não o insere na página. Qual linha faz a inserção? Cite duas vantagens de separar a criação da inserção em funções diferentes.

28. [Código] Modifique `criarComentario` para que cada comentário mostre também a hora em que foi criado, dentro de um elemento `<small>`. Dica: `new Date().toLocaleTimeString("pt-BR")`.

Parte 5 – A função addCommentFromInputBox (linhas 18 a 29)

Esta função valida o texto digitado, cria o comentário e o exibe com animação.

29. [Análise] Explique, linha por linha, o que acontece entre as linhas 19 e 23. Por que o comentário da linha 19 diz que a instrução "bloqueia espaços vazios"?

30. [Teste] O que o `return;` da linha 22 devolve? Remova essa linha, tente adicionar um comentário só com espaços e descreva o resultado. Depois, reescreva as linhas 20 a 28 usando `if/else`, sem `return`. Qual versão você considera mais legível?

31. [Conceitual] Se `texto` já é uma string vazia, por que a linha 21 ainda executa `$input.val("")`? Pense no que estava escrito no campo.

32. [Teste] Explique o papel das linhas 25, 26 e 27 em conjunto. Comente a linha 25, adicione um comentário e descreva a diferença. Por que o `hide()` precisa vir antes do `append()` e do `fadeIn()`?

33. [Conceitual] Qual é a utilidade de `.focus()` na linha 28 para quem está usando a página?

34. [Código] Acrescente uma segunda validação: comentários com mais de 140 caracteres não devem ser adicionados, e o usuário deve receber um aviso com a quantidade de caracteres digitada. Indique em que linha o novo código deve entrar.

Parte 6 – Eventos, callbacks e delegação (linhas 30 a 46)

Nada acontece na página até o usuário clicar ou digitar: é o tratamento de eventos que liga a interface às funções.

36. [Conceitual] Na linha 36, a função passada para `.on("click", ...)` é chamada de *callback*. O que é um callback? Quem chama essa função e em que momento?

37. [Teste] A função da linha 36 recebe o parâmetro `event`, mas não o utiliza. Remova o parâmetro e teste. O código continua funcionando? O que isso revela sobre parâmetros de funções em JavaScript?

38. [Teste] Na linha 40, o que significa `event.keyCode === 13`? Descomente temporariamente as linhas 44 a 46, pressione as teclas Enter, espaço e a letra a no campo de texto e registre os códigos exibidos no console.

39. [Código] O evento `keypress` e a propriedade `keyCode` são considerados obsoletos. Reescreva as linhas 39 a 43 usando o evento `keydown` e a propriedade `event.key`.

40. [Conceitual] O botão `+` e a tecla Enter executam a mesma função, `addCommentFromInputBox`. Que boa prática de programação isso representa? O que aconteceria com a manutenção se o código de adição estivesse copiado dentro de cada evento?

41. [Teste] A linha 30 chama `.on()` com três argumentos.

- a) Explique cada um dos três argumentos.
- b) Substitua temporariamente a linha 30 por `$(".delete-comment").on("click", function(){`, adicione comentários e tente excluí-los. O que acontece?
- c) Explique o resultado usando o conceito de **delegação de eventos** e a ordem em que os elementos passam a existir.

42. [Análise] Na linha 31, a qual elemento `this` se refere? Por que ele é escrito como `$(this)` antes de chamar `.closest()`?

43. [Análise] Na estrutura atual do comentário, `.parent()` funcionaria no lugar de `.closest(".comment")`? E se o botão estivesse dentro de um `<div>` dentro do `<p>`? Qual método é mais robusto e por quê?

44. [Análise] O `this` da linha 32 é o mesmo elemento do `this` da linha 31? Explique. Por que o `.remove()` está dentro da função passada para `fadeOut`, e não logo depois dela?

45. [Teste] Substitua as linhas 31 a 33 por `$(this).closest(".comment").fadeOut().remove();`. Teste a exclusão e explique a diferença de comportamento, usando a ideia de que a animação é **assíncrona**.

Parte 7 – Rastreamento da execução e caça aos erros

Agora o objetivo é enxergar o programa inteiro funcionando e reconhecer o que pode dar errado.

46. [Análise] O usuário abre a página, digita `Oi` e pressiona Enter. Numere os acontecimentos abaixo de 1 a 11, na ordem em que ocorrem.

Ordem	Acontecimento
	`addCommentFromInputBox` lê o campo e remove os espaços das pontas
	O DOM fica pronto e o jQuery executa `main`
	O novo `<p>` é ocultado, inserido em `.comments` e aparece com `fadeIn`
	O arquivo do jQuery é carregado
	O usuário digita `Oi` e pressiona Enter
	Os tratadores de eventos são registrados (linhas 30, 36 e 39)
	O campo é limpo e recebe o foco
	`comentarios.js` é carregado, define `main` e registra-a no `ready`
	`criarComentario` monta o `<p>` com o texto e o botão
	`$input` e `$comments` passam a guardar os elementos da página
	O callback do `keypress` verifica que `keyCode` é 13

47. [Teste] Tente excluir os dois comentários que já vêm escritos no `index.html`. O que acontece? Explique a causa comparando o HTML desses parágrafos com o HTML gerado por `criarComentario`. Proponha duas soluções: uma alterando só o HTML e outra alterando só o JavaScript.

48. [Análise] No `style.css`, a última regra (`delete-comment { ... }`) não tem efeito nenhum na página. Encontre o erro e corrija-o.

49. [Teste] Adicione três comentários e observe as cores de fundo. Em seguida, exclua o segundo comentário da lista. As cores dos comentários seguintes mudam? Explique usando as regras `nth-child(even)` e `nth-child(odd)` do CSS.

50. [Análise] Cada trecho abaixo é uma versão alterada de uma linha do código original. Para cada um, indique a linha original, o que acontece ao executar, a mensagem de erro (se houver) e o tipo de problema: **sintaxe**, **execução**, **lógica** ou **segurança**.

- a) `$(document).ready(main());`
- b) `var texto = $input.val.trim();`
- c) `if(texto = ""){`
- d) `$comments.append(novoComentario);`
- e) `$(".delete-comment").on("click", function(){`
- f) `var $texto = $("<span>").html(texto);`

- g) A chave `}` da linha 23 foi apagada.

Parte 8 – Desafios de modificação e síntese

Os desafios aplicam o que foi estudado; o exercício 55 é a prova de que você entende o arquivo inteiro.

51. [Código] Crie um contador que mostre o texto "N comentário(s)" abaixo do campo de digitação. O número deve ser atualizado sempre que um comentário for adicionado ou excluído. Indique quais linhas do código original você precisou alterar e por quê.

52. [Código] Antes de excluir um comentário, peça confirmação ao usuário com `confirm("Excluir este comentário?")`. Se o usuário cancelar, nada deve acontecer.

53. [Código] Faça o botão + ficar desabilitado enquanto o campo estiver vazio ou contiver só espaços. Dica: use o evento `input` e o método `.prop("disabled", ...)`. Atenção: o botão também precisa voltar a ficar desabilitado depois que um comentário é adicionado. Por quê?

54. [Código] Reescreva `comentarios.js` inteiro em JavaScript puro, sem jQuery (pode deixar de lado as animações). Antes, complete a tabela de equivalências.

jQuery	JavaScript puro
`$(document).ready(fn)`	
`$(".seletor")`	
`$("<p>")`	
`.addClass("x")`	
`.attr("type", "button")`	
`.text(t)`	
`.val()` e `.val("")`	
`.append(a, b)`	
`.on("click", fn)`	
Delegação: `.on("click", ".delete-comment", fn)`	
`.closest(".comment")`	
`.remove()`	

55. [Síntese] Explique o arquivo inteiro com suas palavras. Para cada linha ou bloco, escreva o que a sintaxe significa e qual é o objetivo daquele trecho no funcionamento da aplicação.

Linha(s)	O que a sintaxe significa	Objetivo no programa
1 e 47		

Linha(s)	O que a sintaxe significa	Objetivo no programa
2		
3		
4 e 5		
7		
8		
9		
10 a 13		
14		
15 e 16		
18		
19		
20 a 23		
24		
25 a 27		
28 e 29		
30		
31		
32 a 34		
36 a 38		
39 a 43		
44 a 46		
48		